Static Code Review Report

Employee Management System Version 1.0

Tool: SonarQube | Analysis Date: April 15, 2026

Group 4: Amir Noori, Jesper Holmstrom, Osama Aamer, Mehdi Zaidane

1. Summary Metrics

The table below summarizes all key metrics from the SonarQube analysis of the project.

Lines of Code	5,612	Java-based JavaFX + JPA project
Total Lines (incl. blanks/comments)	7,052	33 files, 34 classes, 454 functions
Statements	3,308	Executable statements
Comment Lines	211 (3.6%)	Below recommended 10–20% threshold
Cyclomatic Complexity	1,289	Total across all methods
Cognitive Complexity	943	Measure of code readability burden
Reliability Issues	1 (Rating C)	DepartmentController.java — 15min effort
Maintainability Issues	163 (Rating A)	Technical debt: 3d 1h, ratio 0.9%
Security Issues	0 (Rating A)	No confirmed open vulnerabilities
Security Hotspots	9 (Rating E)	0% reviewed — Quality Gate failure
Code Coverage	44.2% overall / 48.4% new	Target >= 80% — currently failing gate
Duplicated Lines (overall)	884 (12.5%)	88 blocks, 8 files — above 3% threshold
Duplicated Lines (new code)	241 (7.2%)	23 blocks — failing gate (limit 3%)
Quality Gate	Failed (4 conditions)	New issues, coverage, duplications, hotspots

2. Quality Gate

The project Quality Gate failed with 4 conditions not met on new code since April 9, 2026.

New Issues	0	58 issues	FAILED
Security Hotspots Reviewed	100%	0%	FAILED
Coverage on New Code	>= 80%	48.4%	FAILED
Duplicated Lines on New Code	<= 3%	7.2%	FAILED

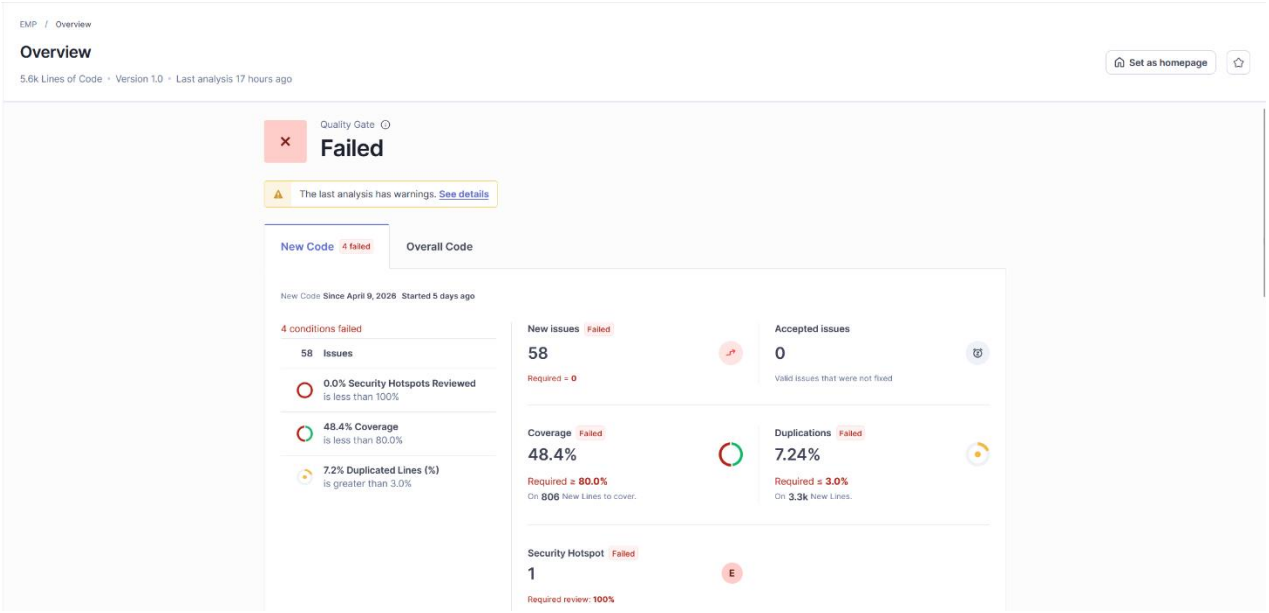


Figure 1: New Code tab — 4 conditions failed, 58 new issues, coverage 48.4%, duplications 7.2%

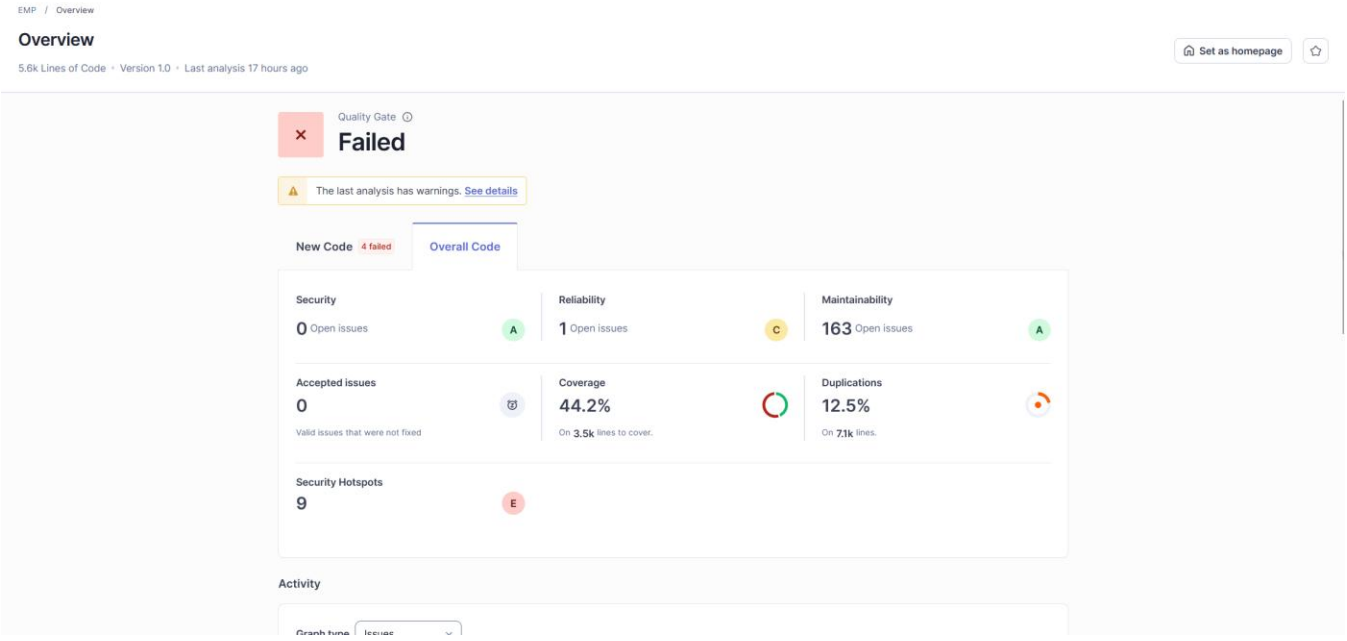


Figure 2: Overall Code — Security A, Reliability C (1 issue), Maintainability A (163 issues), Coverage 44.2%

3. Code Size Metrics

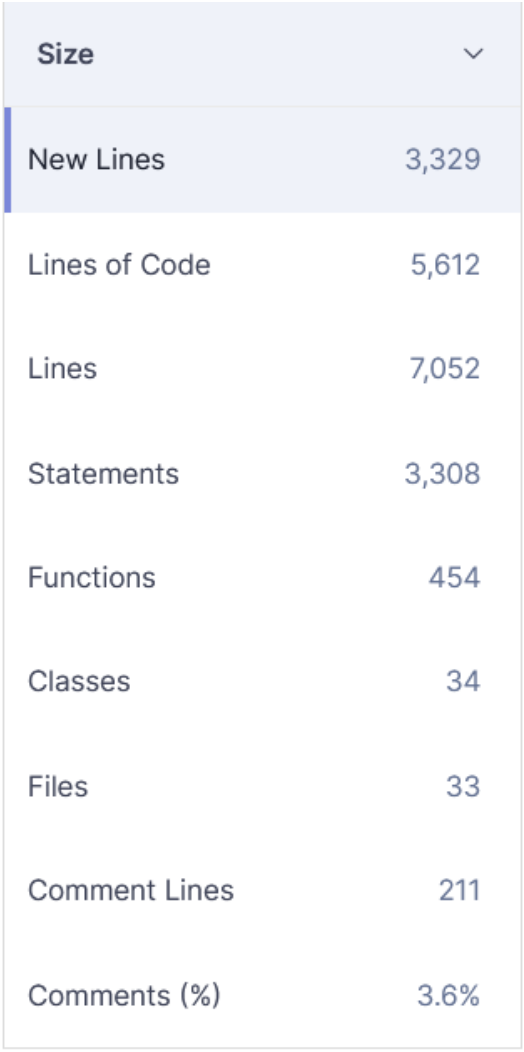


Figure 3: Size metrics — 5,612 LOC, 33 files, 34 classes, 454 functions, 3.6% comments

The project consists of 5,612 lines of code across 33 files and 34 classes. The comment ratio of 3.6% is below the recommended 10–20%, meaning most methods lack documentation. This increases onboarding difficulty and contributes to maintainability issues.

4. Cyclomatic & Cognitive Complexity

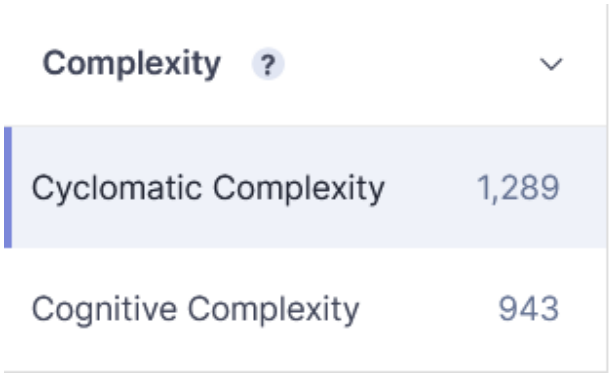


Figure 4: Complexity — Cyclomatic: 1,289 | Cognitive: 943

Cyclomatic complexity of 1,289 across 454 functions gives an average of approximately 2.8 per function, which is within acceptable range. However, individual methods may still exceed the recommended threshold of 10. Cognitive complexity of 943 reflects nesting and control flow difficulty. Methods with cognitive complexity above 15 should be refactored using extract-method or early return patterns.

5. Reliability

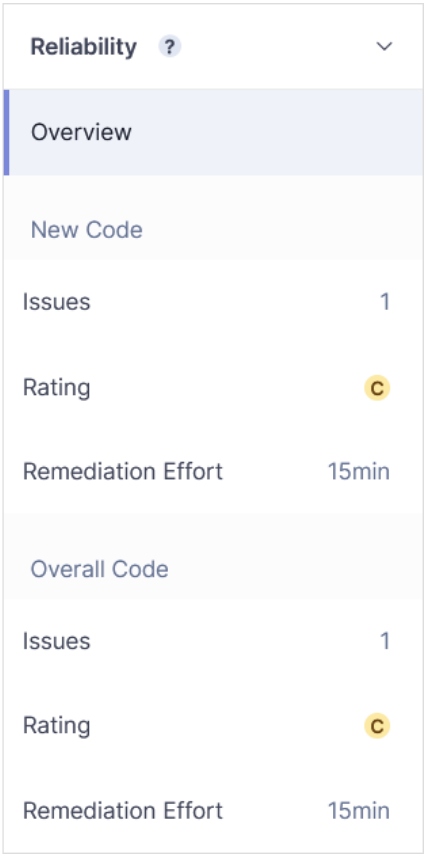


Figure 5: Reliability — 1 issue total, Rating C, 15min remediation effort



Figure 6: Reliability overview — DepartmentController.java (933 LOC, 1 issue, Rating C, 15min effort)

Only one reliability issue was detected, located in DepartmentController.java (933 lines of code). It carries a Reliability rating of C, meaning at least one medium-severity bug is present. The remaining files all achieved Reliability rating A.

6. Maintainability

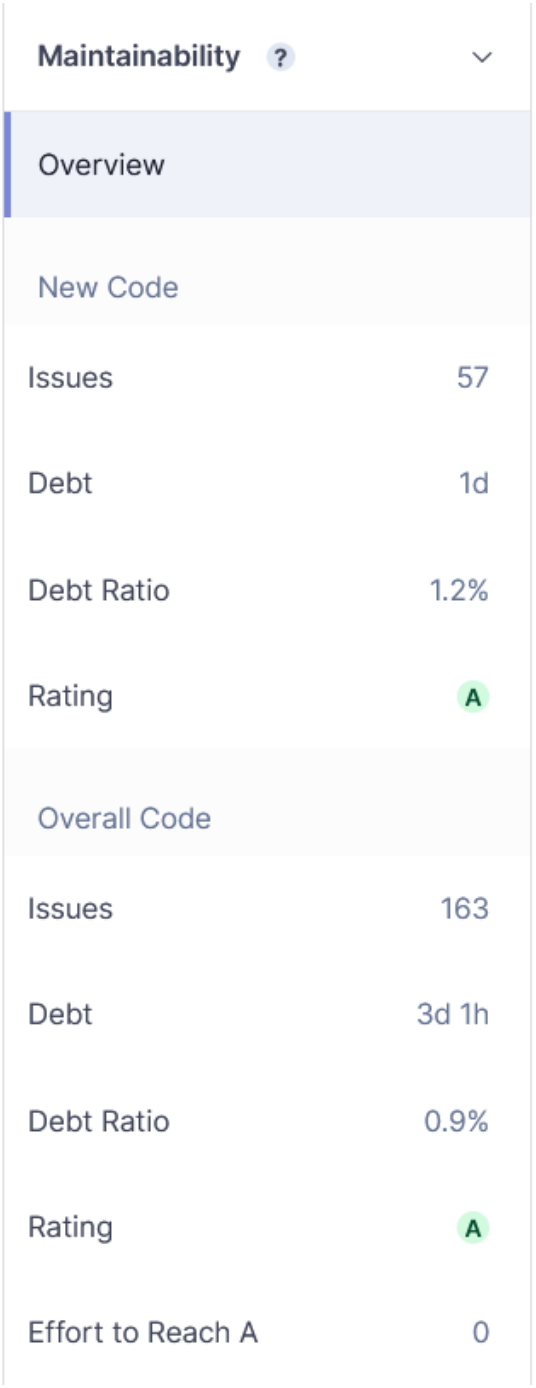


Figure 7: Maintainability — 163 issues, Rating A, 3d 1h debt, 0.9% debt ratio

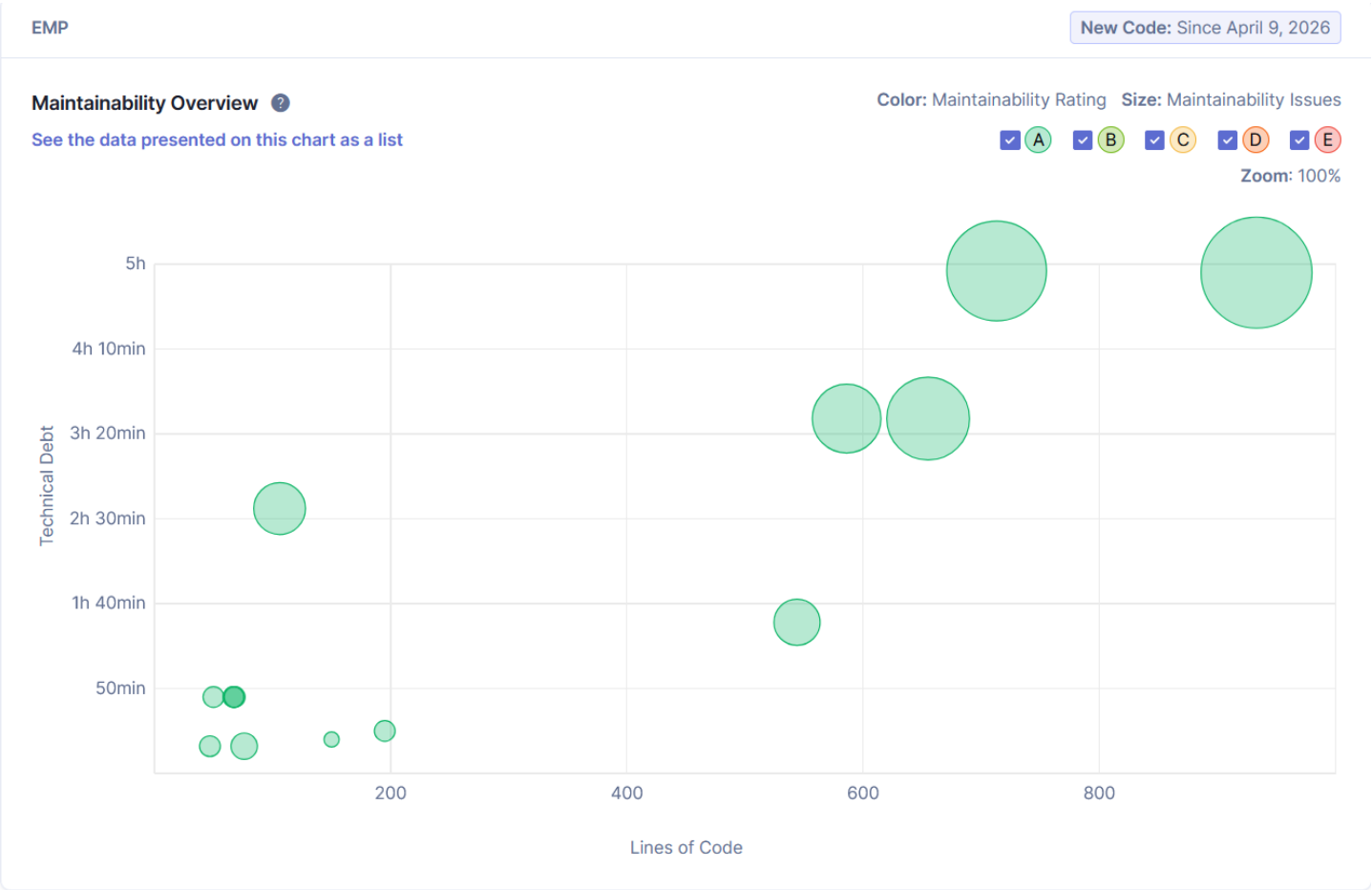


Figure 8: Maintainability overview — larger bubbles = more issues; two files carry 5h+ debt each

The project has 163 maintainability issues with a technical debt of 3 days and 1 hour, giving a debt ratio of 0.9%, within the A-rating threshold. New code introduced 57 additional issues and 1 day of debt in the current sprint. Common issues include missing documentation, overly long methods, and code smells such as unused variables.

7. Code Duplication

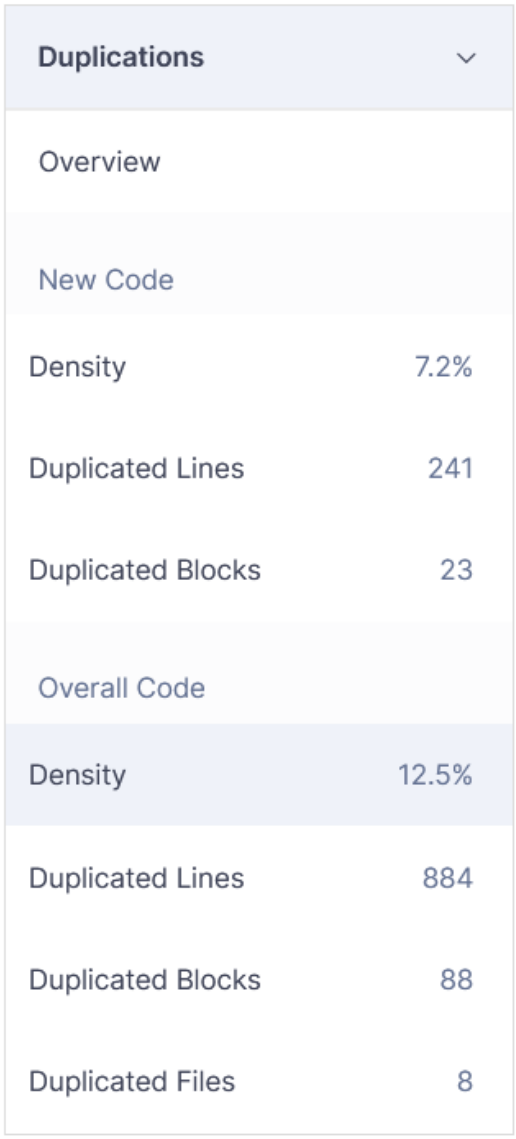


Figure 9: Duplications — overall 12.5% (884 lines, 88 blocks, 8 files); new code 7.2% (241 lines)

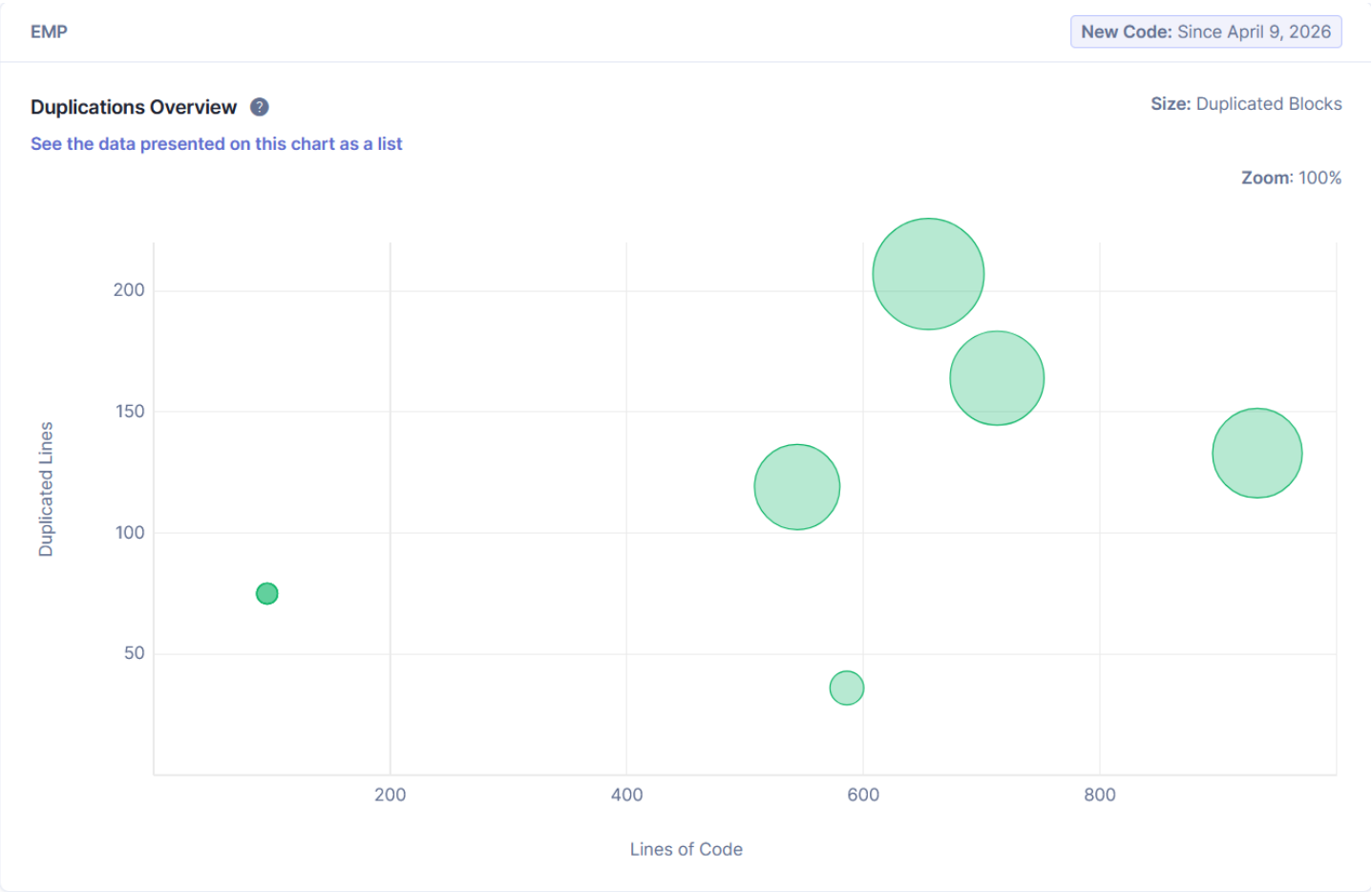


Figure 10: Duplications overview — files with 600–900 LOC carry the most duplicated lines (100–200+)

Overall duplication stands at 12.5%, well above the recommended 5% threshold. On new code specifically, the rate is 7.2%, more than double the Quality Gate limit of 3%. The bubble chart reveals that duplication concentrates in larger files (600–900 LOC range), suggesting that CRUD-pattern DAO classes are repeating similar logic without abstraction. Extracting a generic base DAO class would significantly reduce this metric.

8. Security

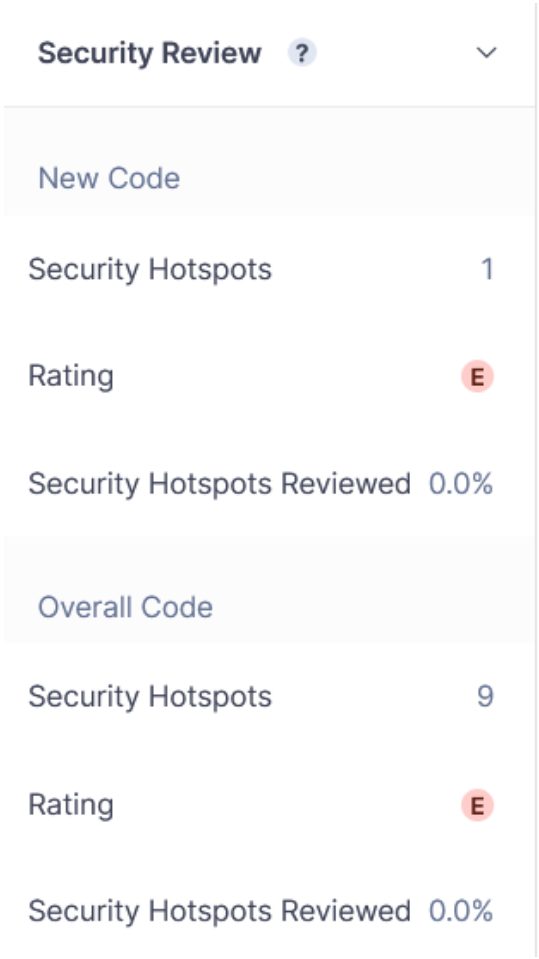


Figure 11: Security Review — 9 hotspots overall, 1 new, 0% reviewed, Rating E

There are no confirmed security vulnerabilities (Rating A). However, 9 security hotspots remain unreviewed, resulting in a Security Review rating of E and a Quality Gate failure. All hotspots must be manually assessed and marked as Safe, Fixed, or Acknowledged before the gate can pass.

9. Key Findings & Recommendations

Priority	Finding	Recommendation
HIGH	4 Quality Gate failures	Resolve new issues, review all 9 hotspots, improve test coverage to 80%, reduce new code duplication below 3%
HIGH	9 unreviewed security hotspots	-
HIGH	Coverage 44.2% (target 80%)	Write JUnit tests for DepartmentController, DAO classes, and service layer methods; integrate JaCoCo in CI
MEDIUM	Reliability issue in DepartmentController	Investigate and fix the single reliability bug
MEDIUM	12.5% code duplication	Extract shared DAO logic into a generic base class; deduplicate repeated CRUD patterns across entity controllers
MEDIUM	163 maintainability issues	Address the 2 files with 5h+ technical debt first; add Javadoc to public methods to improve comment ratio from 3.6%
LOW	High cognitive complexity in some methods	Refactor deeply nested methods using early returns or extract-method; target any method with cognitive complexity > 15

10. Conclusion

The static analysis of the Employee Management System reveals a project with a solid maintainability foundation (Rating A, 0.9% debt ratio) but several areas requiring attention before a production release.

The most critical issues are the four Quality Gate failures: unreviewed security hotspots, insufficient test coverage (44.2% vs. 80% target), excessive new-code duplication (7.2% vs. 3% limit), and 58 newly introduced issues. The single reliability bug in DepartmentController.java should be fixed immediately given its minimal effort. Addressing these items will move the project toward a passing Quality Gate and a more maintainable, secure codebase.